D4 · run/run.sh cheat sheet

`run/run.sh [<runtime>:]<component>:<command> [args] [-- passthrough] [--verbose|-v] [--yes|-y] [--json]`

Runtimes

`local`	a detached process on local Node 24 (pid + log in `run/.run`)
`podman`	container `d4-<name>` on the host network
`(omitted)`	the component's default (`ls` marks it `*`); `test` defaults to local

Infra and the gateway run on podman only. A selection mixing them uses podman for those and says so; naming one alone with `local:` fails. Old `loc`/`con` are rejected with a hint.

Components (selectors)

Selector	Selects
`api`	one component by name
`api,worker`	a list (any mix of names, categories, groups)
`ui`	web, ui-modules, web-explore · -queries · -plan · -data · -reporting · -admin
`service`	api, worker, renderer
`infra`	gateway, postgres, neo4j, redis, s3, smtp, keycloak (+ observability)
`tool`	cli, ops, e2e (run once; no process)
`observability`	group: otel-collector, tempo, prometheus, grafana
`stack`	everything except tools and *optional* ones (web-<module>, observability)

Always in dependency order (stop: reverse). `run/run.sh ls` lists all with category, runtimes and port.

Commands × runtime

Command	local	podman
`build`	turbo build of deps (or package: `build: full`)	turbo build + thin image `localhost/d4-x:<hash>`, skipped if hash exists
`deploy`	build → stop on the other runtime → start → wait healthy → hooks; starts missing deps + gateway	
`start`	like deploy without building (builds only what was never built)	
`stop`	wherever it runs, or only on the runtime given; reverse dependency order	
`reset`	stop, delete `dist/` + local log	stop, remove container **and volumes**; asks unless `--yes`
`status`	table of where/state/health/route/endpoint/build/uptime	
`logs`	`tail -F` of `<name>.local.log`	`podman logs -f`
`test`	turbo on the host	same in pinned Node image, repo mounted
`exec`	command with its env on host	`podman exec`; tools: `podman run --rm`
`shell`	$SHELL with its settings	`podman exec -it … sh`

Without a component: `ls` · `status` · `doctor` · `help [component]`. exec/shell take one component; test one component or `stack`.

Everyday

`podman:stack:deploy`	whole stack; only changed parts rebuild (no-op ≈ 18 s)
`status`	where everything runs (≈ 2 s); `--json`
`local:api:deploy`	API on Node (tsx watch); container stopped
`podman:api:deploy`	…and back (stops the local process)
`local:web:deploy`	shell on Vite, hot reload via the gateway
`local:web-plan:deploy`	one module under `/mfe/plan/`; falls back to ui-modules when stopped
`local:service,ui:deploy`	every app on Node (renderer, ui-modules stay podman)
`podman:service:build`	build service images only
`api:logs`	follow; `-n 50 --no-follow` for a tail
`stack:stop`	stop everything (data kept)
`podman:smtp:reset -y`	recreate one component from scratch
`stack:reset --yes`	**deletes all volumes**, D4 images, run/.run state
`podman:observability:deploy`	OTel, Tempo, Prometheus, Grafana (`/grafana/`)

test

Form	Runs
`api:test`	unit tests of packages changed since HEAD in api's closure
`api:test --all`	every package api depends on
`api:test --base main`	changed since `main` (+ uncommitted)
`worker:test integration`	with Postgres, Neo4j, Redis, S3 (started if needed)
`stack:test static`	typecheck + lint (+ knip for stack)
`web-plan:test e2e`	browser specs mapped in its `test.e2e`
`stack:test e2e`	all specs (≈ 4 min); `-- --grep catalog`
`stack:test smoke`	health, sign-in, `/api/v1/me`
`podman:cli:test --all`	unit tests inside the Node image
`e2e:test e2e -- -c screenshots.config.ts`	regenerate user-manual shots

Root config changes (package.json, lockfile, tsconfig.base, turbo, eslint) select everything in scope. e2e defaults to `--retries=0`.

exec: tools & operator tasks

`cli:exec -- import samples/ubs-banking.yaml -w default`	CLI as dev-admin
`local:cli:exec -- export -w default -o x.yaml`	CLI on Node
`keycloak:exec -- token [user]`	access token
`keycloak:exec -- tenant-user t_x alice [role…]`	Keycloak user
`ops:exec -- tenant create t_x "Name"`	tenant + its Neo4j (7701+)
`ops:exec -- tenant remove t_x`	deregister, stop (volume kept)
`ops:exec -- tenant list · quota t_x k=v`	passthrough to d4-ops
`ops:exec -- seed · schema t_local`	re-run the hooks by hand
`web-plan:exec -- publish 1.2.0 --activate`	module release (`--keep N`, `--dist -`)
`podman:api:exec -- node -v`	in the running container
`postgres:shell`	sh inside `d4-postgres`

status columns

`RUNTIME`	podman · local · - · red `podman+local` = conflict
`STATE`	running · stopped · exited · **conflict** · **orphan** (pid file, no process)
`HEALTH`	live check (≤ 30 s old reused): healthy · unhealthy
`BUILD`	image hash · `(stale)` = source changed · pinned · source (watch) · via ui-modules

Exit code ≠ 0 on conflict, orphan or unhealthy. Filter: `service:status`, `api,web:status`.

Where things live

`host.conf`	D4_SCHEME · D4_HOST · D4_PORT · D4_TLS_PORT · D4_BIND
`component.yaml`	beside each component: ports, health, env, provides, runtimes, hooks, test
`run/env/*.env`	common · images · dev-secrets · local.env (yours) · prod.env
`run/.run/logs/`	run.log, build.log, <name>.{image,package,podman,local,hooks}.log
`run/.run/env/`	merged env per runtime/component (what processes read)
`run/schema/`	component.schema.json (the contract)

Shell vars win: `WEBHOOK_INTERVAL_MS=2000 run/run.sh local:worker:deploy`. `D4_PROFILE=prod` = prod.env, no dev secrets.

Ports (127.0.0.1, same on both runtimes)

gateway 5173/5443	api 3000	worker 3002	renderer 3003
web 8081	ui-modules 8082	web-explore…admin	8091–8096
postgres 5432	neo4j 7687/7474	redis 6379	s3 8333
smtp 1025/8025	keycloak 8080/9000	grafana 3001	tenants 7701+

Routes: / web · /api · /auth · /mfe/<name>/ · /mail/ · /grafana/. Users dev-admin/-viewer/-author/-steward, password d4-local-dev.

When something fails

`status`	first: where it runs, health, conflicts
`doctor`	Podman, Node 24, cgroups, ports, contract, old containers
`… --verbose`	stream every subprocess's output
`<name>:logs`	the component's own output

Every failure prints the command, exit code, last 40 lines, log file and a hint. Build errors stop the run before anything is stopped.